

09 — Codex Skills: installation and use / 09 — Codex Skills 入门与安装使用指南

The question / 问题

How do I find, install, and use a Codex skill on Windows, and how can it help improve a scientific figure without changing the calculation?
在 Windows 上，应该怎样寻找、安装和使用 Codex skill？怎样让它帮助改善科研图的呈现，同时保持计算结果不变？

This guide uses Windows, VS Code, Codex, Anaconda, Python, and Jupyter Notebook. The running example is K-Dense-AI's third-party [scientific-visualization skill](#) ^[1]. Instructions were checked on **September 20, 2026**. Interfaces and discovery paths can change, so the distinctions below matter when revisiting this guide.

本指南面向 Windows、VS Code、Codex、Anaconda、Python 和 Jupyter Notebook 使用环境，以 K-Dense-AI 提供的第三方 [scientific-visualization](#) 科研绘图 skill ^[1] 为贯穿案例。操作说明核对日期为 **2026 年 9 月 20 日**。界面和 skill 的发现路径可能随版本变化，因此以后查阅时，需要留意下文对不同路径的区分。

This chapter is documentation, not an installation record. Writing it does not install the skill, change a Python environment, or execute the plotting exercise. Installation commands below are for a future session when you decide to install it.

本章是操作说明，不是安装记录。 编写本章没有安装这个 skill、修改 Python 环境，也没有执行绘图练习。下文的安装命令供你以后决定安装时使用。

1. What a skill changes / 1. Skill 会怎样影响 Codex 的工作？

A skill gives Codex a reusable procedure for a particular kind of task. Think of a laboratory protocol: it can explain what to inspect, which resources to consult, what steps to follow, and how to check the result. It does not train a new model or guarantee that the output is correct.

Skill 可以理解为给 Codex 的一份可重复使用的操作规程，针对某一类任务说明应该检查什么、查阅哪些资料、按什么步骤操作，以及怎样核对结果。这与实验室里的操作规程（protocol）类似。它不会训练出一个新模型，也不保证每次输出都正确。

For our example, a useful procedure is: inspect the existing plotting code and intended publication size, preserve the data, improve visual presentation, export suitable formats, and inspect the exports. Your prompt supplies the specific file and constraints; the skill supplies reusable guidance.

在本章案例中，一套有用的流程是：检查已有绘图代码与计划发表时的图尺寸，保持数据不变，改善视觉呈现，导出合适的格式，再检查导出结果。你的提示词（prompt）提供具体文件和限制条件；skill 提供可以反复使用的操作指导。

Item 项目	What it provides 提供什么	Example 示例
Ordinary prompt 普通提示词（prompt）	Instructions for the current request 针对本次请求的指令	“Make this legend readable and keep every data point.” “让图例更容易读，同时保留每个数据点。”

Item 项目	What it provides 提供什么	Example 示例
Skill 技能 (skill)	A reusable workflow with optional supporting files 可重复使用的工作流程，以及可选的配套文件	scientific-visualization
Plugin 插件 (plugin)	An installable package that can distribute skills and integrations 可以分发 skills 和集成功能的安装包	A package containing several related workflows 一个包含多个相关工作流程的包
MCP 模型上下文协议 (MCP)	Model Context Protocol: a way to connect an assistant to tools and data Model Context Protocol: 让助手连接工具与数据的一种方式	A server exposing document search 一个向助手提供文档搜索工具的服务
Python library Python 库 (library)	Code executed by the selected Python interpreter 由选定的 Python 解释器 (interpreter) 实际执行的代码	Matplotlib draws the figure; NumPy stores arrays Matplotlib 负责画图; NumPy 用来存储数组

A skill can tell Codex to use Matplotlib, but copying a skill folder does not install Matplotlib. An MCP connection may provide a tool, but it does not by itself specify a complete plotting workflow. Plugin packaging is another distribution route; this chapter teaches installation of one standalone skill folder. See the official [customization overview](#)^[2], [skills and plugins](#)^[3], and [MCP guide](#)^[4].

Skill 可以指导 Codex 使用 Matplotlib，但复制一个 skill 文件夹并不会安装 Matplotlib。MCP 连接可以提供工具，但连接本身不等于一套完整的绘图流程。通过插件打包是另一种分发方式；本章介绍的是安装一个独立的 skill 文件夹。相关概念见官方的[自定义功能概览](#)^[2]、[skills 与插件](#)^[3]以及 [MCP 指南](#)^[4]。

Codex initially sees a skill's name and description, then reads its instructions when selected. A matching task can trigger it implicitly, or you can select it explicitly with \$ or /skills in the Codex interface. These are Codex commands, not PowerShell commands. See [Build skills](#)^[5].

Codex 起初看到 skill 的名称和描述，在选中它时再读取具体指令。符合描述的任务可能触发隐式调用（implicit invocation）；你也可以在 Codex 界面中使用 \$ 或 /skills 显式选择（explicit invocation）。这些内容应输入 Codex，而不是 PowerShell 终端。参见官方 [Build skills](#)^[5]。

2. What is inside a skill folder? / 2. Skill 文件夹里有什么？

The essential file is SKILL.md. The inspected K-Dense example contains these 18 files:

其中必需的文件是 SKILL.md。本次检查的 K-Dense 案例包含下面 18 个文件：

```
scientific-visualization/  
  SKILL.md  
  references/  
    color_palettes.md  
    journal_requirements.md  
    matplotlib_examples.md  
    publication_guidelines.md  
    sources.md  
  scripts/  
    _common.py  
    export_plan.py  
    figure_export.py  
    image_metadata.py  
    palette_audit.py  
    style_presets.py  
    style_preview.py  
  assets/  
    color_palettes.py  
    nature.mplstyle  
    presentation.mplstyle  
    publication.mplstyle  
    publisher_profiles.json
```

Component 组成部分	How to read it 如何理解
SKILL.md	The entry point: metadata, intended use, and workflow instructions 入口文件：包含元数据（metadata）、适用用途和 workflows 说明
references/	Detailed material to consult when relevant, such as figure design guidance 按需查阅的详细参考资料，例如图形设计指导
scripts/	Programs or helpers; inspect their inputs, outputs, and imports before running them 程序或辅助脚本；运行前检查其输入、输出，以及导入的库
assets/	Files used by the workflow, such as templates and style presets 工作流使用的资源文件，例如模板和样式预设
agents/openai.yaml	Optional interface metadata, invocation policy, and declared tool dependencies 可选文件：包含界面元数据、调用策略，以及声明的工具依赖

The top of a SKILL.md usually contains YAML metadata between two --- lines. This teaching example is not a replacement for the upstream file:

SKILL.md 开头通常包含两行 --- 包围的 YAML 元数据。下面是教学示例，不应用它替换作者提供的原始文件：

```
---  
name: scientific-visualization  
description: Improve scientific figures and publication exports.  
---
```

The name identifies the skill. The description helps Codex judge relevance. Instructions below the metadata describe the work. Merely naming a folder correctly is insufficient if its SKILL.md is missing or malformed.

name 用来标识这个 skill; description 帮助 Codex 判断它是否适合当前任务; 元数据之后的正文说明具体做法。如果 SKILL.md 缺失或格式不正确, 即使文件夹名称正确, 也不足以让它正常工作。

The inspected [skill directory](#)^[1] does not contain agents/openai.yaml, requirements.txt, a pyproject.toml, or a dependency lockfile. Its metadata reports version 1.2 and Python 3.11+ compatibility; these are the skill's declarations, not a complete tested environment specification. Keep the folder together: helpers use sibling files and ../assets. Copying only SKILL.md breaks those relationships. Installation also does not turn helpers such as style_presets.py into an importable library in every notebook.

本次检查的 skill 目录^[1]不包含 agents/openai.yaml、requirements.txt、pyproject.toml 或依赖锁定文件 (lockfile)。它的元数据声明版本为 1.2, 兼容 Python 3.11+; 这是 skill 自己的声明, 并不等于一份已经完整测试的环境配置。应保留整个文件夹: 辅助脚本会引用同目录文件以及 ../assets。只复制 SKILL.md 会破坏这些引用关系。安装 skill 也不会让 style_presets.py 之类的辅助脚本自动变成每个 Notebook 都可以导入的 Python 库。

3. Where to find skills and what to check / 3. 去哪里寻找 skill, 先检查什么?

Start with official OpenAI examples and the system skill-installer; third-party GitHub repositories provide additional workflows. An author's repository is useful evidence about that author's code, but it is not an OpenAI endorsement. The official [OpenAI skills repository](#)^[6] also points readers toward current plugin examples.

可以先从 OpenAI 官方示例和系统自带的 skill-installer 开始, 再到第三方 GitHub 仓库寻找其他工作流程。作者的仓库可以帮助你核实其代码, 但不代表 OpenAI 为其背书。官方 [OpenAI skills 仓库](#)^[6]也提供了当前插件示例的入口。

For a candidate, check these items before running its helpers:

找到一个候选 skill 后, 在运行它的辅助脚本之前, 先检查以下事项:

1. **Identity:** confirm the repository owner, exact folder, and SKILL.md name. Similar names can describe different skills.
身份与来源: 确认仓库所有者、具体文件夹和 SKILL.md 中的名称。名称相似的 skill 可能是完全不同的内容。
2. **Scope:** read the description and workflow. Does it address your figure type? Does it assume a different assistant or operating system?
适用范围: 阅读描述和流程。它是否适合你的图形类型? 它是否默认使用另一种助手或操作系统?
3. **Dependencies:** inspect imports and shell commands. Distinguish essential packages from optional examples and unrelated repository tools.
运行依赖 (dependencies): 检查导入语句和终端命令, 区分必需软件包、可选示例依赖, 以及仓库中与当前任务无关的工具。
4. **Effects:** look for files written, external requests, credentials, and commands that install software or change configuration.
操作影响: 检查它会写入哪些文件、是否发起外部请求、是否需要凭据, 以及是否包含安装软件或修改配置的命令。
5. **Version and license:** inspect recent changes and the license. Record a commit or release when reproducibility matters.
版本与许可证 (license): 查看近期改动和许可证。如果需要可复现, 应记录对应的提交 (commit) 或发布版本 (release)。

The K-Dense repository has an MIT license^[7]. Installing a single subfolder does not copy that root license. Keep a source and version record, and preserve applicable notices when redistributing files. The skill also asks users to cite its accompanying paper when used in scientific work; read such behavioral instructions before adopting third-party workflows.

K-Dense 仓库使用 MIT 许可证^[7]。只安装其中一个子文件夹，不会把仓库根目录的许可证一起复制过来。应保留来源和版本记录，重新分发文件时保留适用的声明。这个 skill 还要求在科研工作中使用时引用其配套论文；采用第三方工作流程前，也要阅读这类对使用行为的要求。

There are three separate dependency layers:

这里要区分三层依赖：

Layer 层次	What is needed 需要什么
Reading the skill 读取 skill	Codex and the skill files; no plotting package is required just to read instructions 需要 Codex 和 skill 文件；仅阅读操作说明不需要绘图库
Downloading the skill 下载 skill	The installer needs Python and network access; its Git fallback also needs Git 安装器需要 Python 和网络访问；回退到 Git 下载方式时还需要 Git
Running a figure workflow 执行绘图流程	Packages imported by the selected code, such as NumPy and Matplotlib; additional helpers may need more 需要所选代码实际导入的库，例如 NumPy 和 Matplotlib；其他辅助脚本可能有额外依赖

In this version, palette auditing and export planning mostly use the standard library. Image metadata inspection needs Pillow for raster files and pypdf for PDF files; SVG metadata uses the standard library. Plotly static export is a different workflow involving Kaleido and Chrome. Ordinary Matplotlib SVG/PDF export does not require those tools or TeX. See the [helper scripts](#)^[8].

在本次检查的版本中，配色检查和导出规划主要使用 Python 标准库（standard library）。检查图像元数据时，位图文件需要 Pillow，PDF 文件需要 pypdf，SVG 元数据则使用标准库。Plotly 静态导出属于另一种流程，会涉及 Kaleido 和 Chrome。普通的 Matplotlib SVG/PDF 导出不需要这些工具，也不需要 TeX。具体内容见辅助脚本目录^[8]。

Do not install every dependency from the entire scientific-skills repository for one plotting task. Its broader Python, WSL, and uv setup instructions are not automatically requirements of this single skill. Start with the existing Anaconda environment and inspect the exact helper you intend to use. No account, API key, MCP server, or additional skill was found to be mandatory for this local plotting example.

不要为了一个绘图任务，把整个科研 skills 仓库里的全部依赖都安装一遍。仓库中更广泛的 Python、WSL 和 uv 配置说明，并不自动成为这个单独 skill 的安装要求。先检查现有 Anaconda 环境，以及你真正准备运行的辅助脚本。本次检查没有发现这个本地绘图案例必须使用额外账号、API 密钥、MCP 服务或其他 skill。

4. Choose the installation scope and path / 4. 选择安装范围和位置

Current official documentation uses `.agents/skills` for standalone user and repository skills. The skill-installer bundled on the machine used to prepare this guide instead defaults to `$CODEX_HOME/skills`, normally `~/.codex/skills`. These are different facts: the documented discovery location and a particular installed helper's default destination. Do not assume that every version scans every historical location.

当前官方文档把 `.agents/skills` 列为独立的用户级和仓库级 skill 位置。但编写本指南时，这台电脑自带的 skill-installer 默认使用 `$CODEX_HOME/skills`，通常就是 `~/.codex/skills`。这描述的是两件不同的事：官方文档规定的发现位置，以及某个已安装辅助程序的默认目标位置。不要假定每个版本都会扫描历史上出现过的所有路径。

Scope 安装范围	Windows example Windows 路径示例	Practical effect 实际作用
Personal, current documented location 个人级：当前文档位置	C:\Users\Hainan\.agents\s\skills\scientific-visualization\	Available across this user's projects in the same host environment 在同一运行环境中，可供这个用户的不同项目使用
Project, current documented location 项目级：当前文档位置	<repository>\.agents\s\skills\scientific-visualization\	Travels with this repository if deliberately committed 如果主动提交到 Git，就可以随仓库一起分发
Observed bundled installer default 本机自带安装器的默认位置	C:\Users\Hainan\.codex\s\skills\scientific-visualization\	Actual default of the inspected helper; verify recognition in your client 本次检查的安装器实际使用的默认路径；是否识别仍需在你的客户端核对

Repository discovery scans relevant `.agents/skills` directories from the current working directory up to the repository root. Same-name skills are not merged. Prefer one intentional installation to several competing copies. The authoritative discovery reference is [Build skills](#)^[5].

仓库级发现机制会从当前工作目录向上查找，直到仓库根目录，扫描相关的 `.agents/skills` 目录。同名 skills 不会自动合并。建议明确保留一份准备使用的安装，避免多个副本互相干扰。发现规则以官方 [Build skills](#)^[5] 为准。

In PowerShell, `$env:USERPROFILE` usually identifies your Windows user folder. `$env:CODEX_HOME`, if set, changes the Codex home used by the inspected installer; it does not make `.codex` and `.agents` interchangeable. A remote, container, or WSL session has its own filesystem and home folder.

在 PowerShell 中，`$env:USERPROFILE` 通常表示你的 Windows 用户目录。如果设置了 `$env:CODEX_HOME`，它会改变本次检查的安装器所使用的 Codex 主目录，但这不代表 `.codex` 和 `.agents` 可以互相替换。远程会话、容器（container）或 WSL 会话有各自的文件系统和用户主目录。

5. Install from GitHub, step by step / 5. 从 GitHub 安装：逐步操作

5.1 Open the right input area / 5.1 先分清应该在哪里输入

For natural-language instructions, use the **Codex chat box** in VS Code. For commands labelled **PowerShell**, choose **Terminal** → **New Terminal** and ensure that the terminal profile is PowerShell. Do not type the displayed `PS C:\...>` prompt itself. [Chapter 04](#) explains the current working directory.

自然语言请求输入到 VS Code 中的 **Codex 对话框**。标为 **PowerShell** 的命令，则通过 **Terminal → New Terminal (终端 → 新建终端)** 打开终端，并确认终端配置使用 PowerShell。不要把界面显示的 `PS C:\...>` 提示符也输入进去。当前工作目录的含义见第 04 章。

5.2 Beginner route: ask skill-installer / 5.2 初学者方式：请 skill-installer 安装

Codex chat box — a future installation request:

Codex 对话框——供以后使用的安装请求：

```
$skill-installer
Install only scientific-visualization from:
https://github.com/K-Dense-AI/scientific-agent-skills/tree/main/skills/scientific-visualization

First inspect the source, its dependencies, and the paths
supported by my installed Codex version.
Use a personal installation in my user .agents/skills folder.
Pass that parent folder explicitly as --dest if needed.
Report the exact destination and every file operation.
Do not install Python packages or change my conda environments.
If an existing copy is present, report it without overwriting.
```

```
$skill-installer
请只从下面的位置安装 scientific-visualization:
https://github.com/K-Dense-AI/scientific-agent-skills/tree/main/skills/scientific-visualization

先检查来源、运行依赖，以及本机 Codex 版本支持的路径。
采用个人级安装，放入我的用户目录下的 .agents/skills。
如有必要，将这个父目录显式传给 --dest。
报告准确的安装目标路径，以及每一项文件操作。
不要安装 Python 包，也不要修改我的 conda 环境。
如果已经有一份安装，请报告其位置，不要覆盖。
```

For a project installation, replace the personal-scope instruction with: “Use .agents/skills under the repository root as the explicit destination.” Codex may need approval to write outside the project or download files, depending on your current permission settings.

如果希望项目级安装，把个人级安装的那句改成：“明确使用仓库根目录下的 .agents/skills 作为目标位置。”根据当前权限设置，Codex 在向项目之外写入文件或下载文件时，可能需要操作批准。

5.3 Explicit PowerShell route / 5.3 明确执行 PowerShell 命令的方式

The commands below use the locally available installer. Its location can differ on another machine. If `Test-Path` reports `False`, ask Codex to locate its installed `skill-installer` rather than downloading random replacement scripts.

下面的命令使用本机已有的安装器。在另一台电脑上，安装器的位置可能不同。如果 `Test-Path` 返回 `False`，请让 Codex 查找它已安装的 `skill-installer`，不要随意下载其他脚本代替。

PowerShell — inspect paths; no skill is installed yet:

PowerShell 终端——检查路径；这一步还没有安装 skill:

```

Get-Location
python -c "import sys; print(sys.executable)"
$skillCodexHome = $env:CODEX_HOME
if (-not $skillCodexHome) {
    $skillCodexHome = Join-Path $env:USERPROFILE '.codex'
}
$installer = Join-Path $skillCodexHome (
    'skills\.system\skill-installer\scripts\' +
    'install-skill-from-github.py'
)
Test-Path -LiteralPath $installer
python -B $installer --help

```

If python is unavailable, use the existing Anaconda PowerShell Prompt or the known full path to your Python executable. Selecting an interpreter in VS Code does not prove an already-open terminal uses it; check `sys.executable` as above.

如果系统无法识别 `python`，可以打开现有的 Anaconda PowerShell Prompt，或者使用已知的 Python 可执行文件完整路径。在 VS Code 中选择了解释器，并不能证明已经打开的终端也在使用同一个解释器；应按上面的方式检查 `sys.executable`。

PowerShell — choose one destination:

PowerShell 终端——选择一个目标位置：

```

# Personal installation:
$skillParent = Join-Path $env:USERPROFILE '.agents\skills'

```

For a project installation, open the repository folder in VS Code, verify its root, then use this alternative assignment:

如果选择项目级安装，先在 VS Code 中打开目标仓库文件夹，核对仓库根目录，再改用下面这段赋值：

```

$projectRoot = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Open the intended Git repository.' }
$skillParent = Join-Path $projectRoot '.agents\skills'

```

`--dest` receives the **parent skills directory**. The installer adds `scientific-visualization` beneath it. Giving the final skill directory as `--dest` would create an unnecessary nested directory.

`--dest` 接收的是 **skills 的父目录**。安装器会在其下创建 `scientific-visualization`。如果把最终的 skill 文件夹也写进 `--dest`，就会多嵌套一层目录。

PowerShell — installation command; downloads and writes files:

PowerShell 终端——安装命令；这一步会下载并写入文件：

```

$installArgs = @(
    '--repo', 'K-Dense-AI/scientific-agent-skills',
    '--path', 'skills/scientific-visualization',
    '--ref', 'main',
    '--dest', $skillParent
)
python -B $installer @installArgs
if ($LASTEXITCODE -ne 0) { throw 'Skill installation failed.' }

```

If you change `$skillParent`, rebuild `$installArgs` before rerunning: an existing array retains the old destination value. The command is an alternative to the chat-box route; do not run both to create duplicate installations.

如果修改了 `$skillParent`，再次运行之前要重新执行构建 `$installArgs` 的那段代码，因为已经创建的数组仍保存旧的目标路径。终端命令方式与前面的对话框方式任选其一即可，不要两种都执行而产生重复安装。

The helper also accepts `--url` with the GitHub folder URL. In the inspected implementation, a URL containing `/tree/main/` supplies its own reference; adding `--ref` does not override that main. For a pinned version, use `--repo` plus `--path` and replace main with a real reviewed commit or tag. Do not copy an invented commit identifier.

安装器也支持通过 `--url` 传入 GitHub 文件夹链接。在本次检查的实现中，包含 `/tree/main/` 的 URL 自身已经指定了引用版本，另加 `--ref` 不会覆盖 URL 里的 main。如果需要固定版本（pin a version），使用 `--repo` 加 `--path`，并把 main 换成你实际检查过的提交或标签（tag）。不要复制虚构的提交编号。

5.4 What does installation change? / 5.4 安装具体会修改什么？

For the inspected installer, installation copies the selected skill directory and its files into the destination. It creates missing destination parents. The download route temporarily downloads the repository ZIP, then copies only the requested skill. Temporary work normally lives under `%TEMP%\codex\skill-install-*`; the actual location follows Python's temporary-directory setting, and cleanup is attempted afterwards. An existing destination is an error, not an automatic update. The `-B` flag above prevents Python bytecode cache files in the installer's own directory.

对于本次检查的安装器，安装操作会把选中的 skill 目录及其文件复制到目标位置，并创建尚不存在的上级目录。下载方式会先临时下载仓库 ZIP，然后只复制指定的 skill。临时工作文件通常位于 `%TEMP%\codex\skill-install-*` 下；实际位置取决于 Python 的临时目录设置，结束后会尝试清理。如果目标文件夹已经存在，安装器会报错，而不是自动更新。上面命令中的 `-B` 用来避免在安装器自身目录中产生 Python 字节码缓存文件。

This file-copy operation does not itself run the downloaded helper scripts, install Python packages, change conda environments, modify the plotting script, edit `config.toml` or `AGENTS.md`, or register an MCP server. Project installation adds files that Git may show as untracked; it does not commit or push them. Temporary cleanup may be incomplete after an interrupted process. These details were checked against the local installer and can be rechecked in the [installer source](#) ^[9].

这项文件复制操作本身不会运行刚下载的辅助脚本、安装 Python 包、修改 conda 环境、改动绘图脚本、编辑 `config.toml` 或 `AGENTS.md`，或者注册 MCP 服务。项目级安装会新增文件，Git 可能把它们显示为未跟踪（untracked），但不会自动提交或推送。如果进程被中断，临时文件可能没有完全清理。这些细节已对照本机安装器核对，也可以在安装器源代码 ^[9] 中再次检查。

6. Confirm recognition and use the skill / 6. 确认识别并调用 skill

First verify the files. Continue in the same PowerShell session, or set `$skillParent` again if you opened a new terminal:

先核对安装文件。继续使用同一个 PowerShell 会话；如果重新打开了终端，要先重新设置 `$skillParent`：

```
$skillFolder = Join-Path $skillParent 'scientific-visualization'
Test-Path -LiteralPath (Join-Path $skillFolder 'SKILL.md')
Get-ChildItem -LiteralPath $skillFolder
```

Then type \$ in the **Codex chat box** and look for the skill, or use /skills if available in your interface. Newly installed skills should be discovered automatically; if the list remains stale, start a new Codex conversation or restart the extension. Saving a skill file is not proof that the current session has selected it.

然后在 **Codex 对话框**里输入 \$，查看候选列表中是否有这个 skill；如果当前界面支持，也可以使用 /skills。新安装的 skills 应自动被发现；如果列表仍未更新，可以新建 Codex 对话或重启扩展。文件保存成功，并不等于当前会话已经选中这个 skill。

The inspected installer's list-skills.py checks its default \$CODEX_HOME/skills directory for installed annotations. A skill deliberately placed in .agents/skills may therefore lack that annotation. Check Codex's actual skill selector and loaded path instead of treating the installer's list as the discovery result.

本次检查的安装器中，list-skills.py 根据默认的 \$CODEX_HOME/skills 目录判断是否添加“已安装”标记。因此，明确安装在 .agents/skills 中的 skill 可能没有这个标记。应检查 Codex 实际的 skill 选择列表和加载路径，不要把安装器列出的状态直接当作 Codex 的发现结果。

Codex chat box — verify without changing your project:

Codex 对话框——在不修改项目的情况下验证：

```
$scientific-visualization
Read this skill and report the SKILL.md path you loaded.
Summarize how it would review a Matplotlib figure.
List any dependencies needed for the proposed workflow.
Do not edit files, install packages, or run plotting scripts.
```

```
$scientific-visualization
请读取这个 skill，并报告实际加载的 SKILL.md 路径。
概述它会怎样检查一张 Matplotlib 图。
列出计划采用的流程需要哪些依赖。
不要修改文件、安装软件包或运行绘图脚本。
```

For implicit use, a request such as “Improve this Matplotlib figure for a journal submission” may match the description. For a learning exercise, use the explicit name and ask for the loaded path. Availability, selection, and successful execution are three different checks.

对于隐式调用，“请优化这张用于期刊投稿的 Matplotlib 图”之类的请求，可能会匹配这个 skill 的描述。在学习练习中，建议明确写出名称，并要求报告加载路径。“可被发现”“已被选中”“执行成功”是三个不同的检查环节。

Symptom 现象	What to inspect 检查什么
Skill missing from the selector 选择列表里找不到 skill	Destination, SKILL.md, metadata, active workspace, and client version; then refresh 检查安装位置、SKILL.md、元数据、当前工作区和客户端版本，然后刷新
Available but not selected automatically 可以找到，但没有自动选中	Task-description match and policy.allow_implicit_invocation in agents/openai.yaml, if present; try explicit invocation 检查任务是否匹配描述；如有 agents/openai.yaml，检查其中的 policy.allow_implicit_invocation，然后尝试显式调用

Symptom 现象	What to inspect 检查什么
Two similar skills appear 出现两个相似的 skill	Duplicate names across personal, project, or plugin locations; inspect actual paths 检查个人级、项目级或插件位置是否有同名副本，并核对实际路径
Skill is selected but plotting fails 选中了 skill，但绘图失败	Python interpreter, imports, fonts, data paths, and write permissions 检查 Python 解释器、导入的库、字体、数据路径和写入权限
Installer reports an existing destination 安装器提示目标已存在	Treat it as an update decision; inspect and back up the existing copy 按更新操作处理，先检查并备份已有副本
Works in a terminal but fails in a notebook 终端能运行，Notebook 却失败	The notebook may use a different Python kernel Notebook 可能使用了另一个 Python 内核 (kernel)
Works on Windows but is missing remotely Windows 中可用，远程会话却找不到	Installations belong to the filesystem where Codex is running 安装文件属于 Codex 实际运行位置的文件系统

Jupyter Notebook — run in a Python code cell:

Jupyter Notebook——在 Python 代码单元格 (code cell) 中运行:

```
import sys
print(sys.executable)
import numpy
import matplotlib
print(numpy.__version__, matplotlib.__version__)
```

Compare the interpreter path with the terminal check. Select the intended notebook kernel if they differ. If imports fail, identify the correct environment before considering package installation; the skill folder cannot fix a missing library. See the [VS Code notebook guide](#) ^[10].

把这里的解释器路径与终端检查结果比较。如果不同，为 Notebook 选择你准备使用的内核。如果导入失败，先确认正确的环境，再考虑是否需要安装包；skill 文件夹本身不能补上缺失的 Python 库。参见 [VS Code Notebook 指南](#) ^[10]。

7. Update, disable, or uninstall / 7. 如何更新、禁用和卸载

7.1 Update deliberately / 7.1 有计划地更新

An installed skill does not automatically follow its upstream GitHub source. Pulling this handbook does not update a personal installation; a project copy tracked by this repository can change when pulled commits change it. The installer has no overwrite/update switch in the inspected version.

已安装的 skill 不会自动跟随其上游 GitHub 来源更新。拉取本手册不会更新个人级安装；如果项目级副本已由本仓库跟踪，而拉取的提交修改了它，那么这个项目副本可以随之改变。本次检查的安装器版本没有覆盖或更新开关。

1. Identify the exact active skill path and record its source and current version.
确认当前实际生效的 skill 路径，并记录来源和当前版本。

2. Copy your current folder, including local edits, to a backup **outside every scanned skills directory**.
把当前文件夹连同本地修改一起备份，备份位置应在**所有被扫描的 skills 目录之外**。
3. Download the reviewed new version into a separate staging folder using `--dest`.
通过 `--dest`，把已经检查过的新版本下载到单独的暂存文件夹（staging folder）。
4. Compare the old and new `SKILL.md`, scripts, and dependencies. After review, move the old active folder outside the discovery locations and put the reviewed new folder at its original path. Replacing the folder, rather than merging its contents, avoids retaining obsolete helper files.
比较新旧版本的 `SKILL.md`、脚本和依赖。检查完成后，把旧的生效文件夹移到所有发现位置之外，再把已检查的新文件夹放到原来的位置。整体替换文件夹可以避免合并新旧内容时残留过期的辅助文件。
5. Refresh Codex and repeat the read-only recognition check. Keep the backup until your normal workflow succeeds.
刷新 Codex，重新执行前面的只读识别检查。在日常流程验证成功之前，保留备份。

Do not leave an “old” renamed folder containing `SKILL.md` under a scanned directory: it may still be discovered. Updating a skill does not require updating every Python package.

不要只把旧文件夹改名为“old”后仍留在扫描目录中；只要里面还有 `SKILL.md`，它就可能继续被发现。更新 skill 也不意味着必须更新所有 Python 包。

7.2 Disable without deleting / 7.2 禁用，但暂不删除

The official Build skills page shows a `[[skills.config]]` entry pointing to `SKILL.md`. The configuration reference describes the path as a **skill directory**. Because these current descriptions differ, use the documented example first and verify the result in your installed version; do not claim disabling worked merely because the file was saved.

官方 Build skills 页面给出的 `[[skills.config]]` 示例指向 **SKILL.md 文件**，而配置参考页把这个路径描述为 **skill 目录**。由于当前两处描述存在差异，可以先按文档示例操作，再在已安装的版本中验证效果；不要仅凭配置文件保存成功就认定禁用已经生效。

TOML configuration file — edit the effective Codex `config.toml`, not the terminal:

TOML 配置文件——编辑实际生效的 Codex `config.toml`，不要把下面内容输入终端：

```
[[skills.config]]
path = "C:/Users/Hainan/.agents/skills/scientific-visualization/SKILL.md"
enabled = false
```

Use your actual installed path. The usual configuration location is `C:\Users\Hainan\.codex\config.toml`; a custom `CODEX_HOME` can change it. Preserve existing settings and edit an existing matching entry instead of repeatedly appending duplicates. Restart Codex and verify the skill is unavailable. If the installed version expects a directory path, follow its schema and verify again. See [Build skills](#) ^[5] and [configuration reference](#) ^[11].

路径应换成你实际安装的位置。配置文件通常位于 `C:\Users\Hainan\.codex\config.toml`；自定义 `CODEX_HOME` 可能改变这个位置。保留原有设置；如果已经有对应条目，应修改它，不要反复追加重复条目。重启 Codex，并确认 skill 已不可用。如果已安装版本要求填写目录路径，就遵循该版本的配置规则（schema），再验证一次。参见 [Build skills](#) ^[5] 和 [配置参考](#) ^[11]。

A reversible alternative is moving the entire skill folder outside all discovery locations, then refreshing Codex. This is different from disabling only automatic invocation: a skill can remain explicitly callable even when its implicit-invocation policy is off.

另一种可恢复的方式，是把整个 skill 文件夹移到所有发现位置之外，再刷新 Codex。这与仅关闭自动调用不同：即使隐式调用策略已关闭，skill 仍可能允许你显式调用。

7.3 Uninstall / 7.3 卸载

Locate the same verified folder in File Explorer, back up any local edits, and remove **only that skill folder**. Remove its obsolete matching configuration entry if you created one. Refresh Codex and confirm it is absent. Check for another copy if it still appears. Do not delete the entire .agents or .codex directory.

在文件资源管理器（File Explorer）中找到已经核对过的那个文件夹，备份其中的本地修改，然后**只删除这个 skill 文件夹**。如果之前添加过相应配置条目，也移除已经失效的对应条目。刷新 Codex，确认列表中已没有它；如果还在，检查是否有另一份副本。不要删除整个 .agents 或 .codex 目录。

Uninstalling the skill leaves your plots, Python packages, and conda environments in place. Removing a project skill is a repository change to review with `git status`; it is not automatically committed.

卸载 skill 后，你的绘图文件、Python 包和 conda 环境仍会保留。删除项目级 skill 属于仓库改动，应通过 `git status` 检查；它不会自动提交。

8. Practical example: improve a figure without changing the science / 8. 实际例子：改善科研图，保持计算结果不变

The existing [plotting example](#) compares the local PCG64 generator with the global MT19937 generator. It imports data-generation functions from [example 01](#). Both use seed 5, but they produce different streams. Preserving the seed alone is insufficient: preserve the generator type, draw order, parameters, and resulting arrays as well.

手册已有的绘图示例比较局部 PCG64 随机数生成器与全局 MT19937 随机数生成器，并从示例 01 导入数据生成函数。它们都使用随机种子（seed）5，但产生的随机序列不同。因此，只保留随机种子还不够；还要保留生成器类型、抽样顺序、参数和生成的数组。

8.1 A reusable request / 8.1 可以反复使用的请求

Codex chat box — use after the skill is installed:

Codex 对话框——安装好 skill 后使用：

\$scientific-visualization

Improve the scientific presentation of:
examples/02_plot_generated_data.py

Read the skill and report its loaded path.
Inspect the script and its imported data-generation functions.
Keep those functions, both generator types (PCG64/MT19937),
seed 5, draw order, N_TRAIN=10, and NOISE_STD=4*0.03 unchanged.
Preserve $\exp(-x/2)$, the 300-point curve grid, and every array.
Do not smooth, filter, resample, refit, or remove observations.

Improve only fonts, colors, markers, legend placement, spacing,
and export settings. Keep titles and labels scientifically true.
Use a readable font available on this Windows machine.
Use color plus marker or line distinctions for grayscale use.
Keep a clear two-panel comparison at about 7 inches wide.
Avoid clipped labels and legends overlapping observations.

Preserve the existing PNG output and its destination.
Additionally export:
docs/images/09-generated-data-styled.svg
docs/images/09-generated-data-styled.pdf
Do not install packages or modify the Python environment.

Before editing, capture the arrays used in each panel and curve.
After editing, compare old and new values, shapes, and dtypes.
Require exact equality for this deterministic styling-only task.
Check the exported figures visually at the intended print size.
Report edits, output paths, comparisons, and any unavailable checks.
Preserve unrelated work; do not commit or push.

\$scientific-visualization

请优化下面脚本生成的科研图：
examples/02_plot_generated_data.py

读取 skill，并报告实际加载路径。
检查脚本，以及它导入的数据生成函数。
保持这些函数、两种生成器类型 (PCG64/MT19937)、
随机种子 5、抽样顺序、N_TRAIN=10 和 NOISE_STD=4*0.03 不变。
保持 $\exp(-x/2)$ 、包含 300 个点的曲线网格，以及所有数组不变。
不要平滑、过滤、重采样、重新拟合或删除观测值。

只改进字体、配色、标记、图例位置、间距和导出设置。
保持标题和标签在科学含义上准确。
使用本机 Windows 已有且易读的字体。
除了颜色，还要用标记或线型区分数据，便于灰度阅读。
保留清晰的双面板对照，总宽度约为 7 英寸。
避免标签被裁切，也不要让图例遮挡观测值。

保留已有的 PNG 输出及其保存位置。
额外导出：
docs/images/09-generated-data-styled.svg
docs/images/09-generated-data-styled.pdf
不要安装软件包或修改 Python 环境。

修改前，记录每个面板和曲线实际使用的数组。
修改后，比较新旧数组的数值、形状 (shape) 和数据类型 (dtype)。
本任务只调整确定性绘图样式，因此要求数据完全相同。
按预定打印尺寸检查导出图的实际视觉效果。
报告修改、输出路径、比较结果，以及无法执行的检查。
保留无关改动；不要提交或推送。

This prompt is a future exercise, not a claim that the figure has already been changed or exported. If you want to preserve the PNG's appearance too, ask Codex to create a separate plotting script instead of changing the existing script.

这段提示词是供以后操作的练习，并不表示图片已经修改或导出。如果还希望原 PNG 的外观也保持不变，应要求 Codex 新建单独的绘图脚本，而不是修改已有脚本。

8.2 What the plotting changes can look like / 8.2 绘图修改可以是什么样？

The following is a small **Python fragment**, to adapt inside the plotting code after a figure named `fig` exists and before `plt.close(fig)`. It is not a complete program. `output_dir` is deliberately explicit because a notebook's working directory may differ from the script directory:

下面是一个简短的 **Python 代码片段**，需要结合已有绘图代码调整，放在名为 `fig` 的图对象创建之后、`plt.close(fig)` 之前。它不是可独立运行的完整程序。这里明确写出 `output_dir`，因为 Notebook 的当前工作目录可能与脚本目录不同：

```
from pathlib import Path

# Replace this path with your intended existing output directory.
output_dir = Path("docs/images")
if not output_dir.is_dir():
    raise FileNotFoundError("Check the working directory first.")

for ax in fig.axes:
    ax.tick_params(labelsize=9)
    ax.xaxis.label.set_size(10)
    ax.yaxis.label.set_size(10)
    ax.title.set_size(11)

for suffix in ("svg", "pdf"):
    target = output_dir / f"09-generated-data-styled.{suffix}"
    fig.savefig(target, bbox_inches="tight", facecolor="white")
```

Matplotlib determines the output format from the extension here. SVG and PDF support vector elements, but raster images inside a figure remain raster. Increasing DPI does not make text larger. Inspect at the final physical size and check the publication's font and export requirements. `bbox_inches="tight"` can change the final bounding dimensions, so omit it when exact physical dimensions must be preserved. See [Matplotlib savefig](#) ^[12].

这里 Matplotlib 根据文件扩展名决定输出格式。SVG 和 PDF 支持矢量元素（vector elements），但图中嵌入的位图（raster image）仍然是位图。提高 DPI 不会让文字变大。应按最终物理尺寸检查图形，并核对出版物对字体和导出的要求。`bbox_inches="tight"` 可能改变最终的外框尺寸；如果必须保持精确的物理尺寸，就应省略它。参见 [Matplotlib savefig](#) ^[12]。

For editable SVG text, `svg.fonttype="none"` retains text but depends on fonts available to the viewer; `"path"` preserves glyph outlines instead. For PDF, `pdf.fonttype=42` is a possible TrueType setting. Choose deliberately for the publisher's requirements. If changing to `layout="constrained"`, remove conflicting `tight_layout()` calls. See [Matplotlib fonts](#) ^[13] and [constrained layout](#) ^[14].

如果希望 SVG 中的文字可以编辑，`svg.fonttype="none"` 可以保留文本，但显示效果依赖查看者电脑上的字体；`"path"` 则保存字形轮廓。对于 PDF，`pdf.fonttype=42` 是一种 TrueType 字体设置。应根据出版方要求选择。如果改为使用 `layout="constrained"`，要移除与之冲突的 `tight_layout()` 调用。参见 [Matplotlib 字体说明](#) ^[13] 和 [约束布局说明](#) ^[14]。

8.3 Check the result / 8.3 怎样核对结果?

- Compare the actual old and new arrays with `numpy.testing.assert_array_equal`; matching shapes alone are insufficient. Also check dtypes explicitly. Comparing a new array only with its own copy does not test the edit.
使用 `numpy.testing.assert_array_equal` 比较修改前后真正参与绘图的数组；仅检查形状相同还不够，也要明确检查数据类型（dtype）。只把新数组与它自己的副本比较，不能验证这次修改是否改变了数据。
- Open both exports and inspect labels, legends, panel spacing, missing glyphs, and grayscale distinctions. A file existing on disk is only the first check.
打开两种导出文件，检查标签、图例、面板间距、缺失字形，以及灰度下是否还能区分数据。磁盘上存在文件，只是第一项检查。
- Review the code diff: changes should concern presentation and exports. Explain any inability to reproduce the original environment before attributing numerical differences to styling.
查看代码差异（diff）：改动应与呈现和导出有关。如果不能复现原来的运行环境，应先说明限制，不要直接把数值差异归因于样式调整。
- In a notebook, restart the kernel and run relevant cells in order when checking reproducibility; stale variables can hide a changed calculation.
在 Notebook 中检查可复现性时，应重启内核，并按顺序运行相关单元格；旧变量可能掩盖计算已经发生变化的问题。

The expected outcome is the same scientific data with clearer presentation and two additional export files. The exercise passes only after both numerical and visual checks succeed.

预期结果是：科学数据保持不变，图形呈现更清楚，并多出两个导出文件。只有数值检查和视觉检查都通过，这次练习才算完成。

9. Prompt archive: the request behind this chapter / 9. 提示词存档：本章的原始请求

The following is an English translation of the original request, retained for reuse. Its instruction to provide a reply first records the original request; the later instruction to create chapter 09 authorized the handbook files. Neither request authorized installing the example skill.

下面先保留原始请求的英文译文，再保留中文原文，方便以后复制使用。其中“先在回复中给出”的要求记录了最初的请求；后来用户明确要求创建第 09 章，才授权写入手册文件。这两次请求都没有授权安装案例中的 skill。

Codex chat box — guide-authoring prompt:

Codex 对话框——用于编写本指南的提示词：

Please write a beginner's guide to installing and using Codex Skills, for learning and future reference.

I use Windows + VS Code + Codex, as well as Anaconda, Python, and Jupyter Notebook. I am learning these tools. Explain plainly and concretely; do not assume I know the CLI.

Use scientific-visualization as a running example and explain:

1. What a skill is, how it affects Codex, and how it differs from prompts, plugins, MCP, and Python libraries.
2. The roles of SKILL.md, references, scripts, and assets.
3. Where to find skills and check sources, scope, dependencies.
4. Personal versus project installation, locations, GitHub installation with skill-installer, and all file changes.
5. Recognition, explicit and automatic invocation, and troubleshooting when the skill does not take effect.
6. Updating, disabling, and uninstalling.
7. A practical Matplotlib request to improve fonts, colors, legends, and layout while preserving calculated results and exporting SVG/PDF.

Example source:

<https://github.com/K-Dense-AI/scientific-agent-skills/tree/main/skills/scientific-visualization>

Check current official documentation, especially differences between .agents/skills and .codex/skills across versions. Label commands as Codex chat-box input or PowerShell input.

Explain in Chinese with English technical terms.

Use personal-handbook Markdown, steps, comparison tables, and copyable examples suitable for US Letter printing. First provide the complete introduction in the reply. Only write the tutorial: do not actually install a skill or modify environments or project files.

Also include this prompt in the document and assign the next number in the handbook sequence.

请帮我编写一份《Codex Skills 入门与安装使用指南》，供我学习和以后查阅。

我的使用环境是 Windows + VS Code + Codex，也使用 Anaconda、Python 和 Jupyter Notebook。我正在学习这些工具，希望说明通俗、具体，不要默认我熟悉命令行。

请以 scientific-visualization 科研绘图 skill 为贯穿案例，介绍：

1. Skill 是什么？它如何影响 Codex 的工作？与普通提示词、插件、MCP、Python 库分别有什么区别？
2. Skill 的文件结构：SKILL.md、references、scripts、assets 各自有什么作用？
3. 去哪里寻找 skill，怎样检查来源、用途和运行依赖？
4. 完整安装流程：个人级与项目级安装的区别、安装位置、如何使用 skill-installer 从 GitHub 安装，以及安装过程具体会修改哪些文件。
5. 安装后如何确认已识别、如何显式调用、何时会自动调用，以及不生效时如何排查。
6. 如何更新、禁用和卸载 skill。
7. 用一个实际例子说明：如何要求 Codex 优化 Matplotlib 科研图的字体、配色、图例和布局，保持计算结果不变，并导出 SVG/PDF。

案例来源：

[<https://github.com/K-Dense-AI/scientific-agent-skills/tree/main/skills/scientific-visualization>]
(<https://github.com/K-Dense-AI/scientific-agent-skills/tree/main/skills/scientific-visualization>)

请核对当前官方文档，特别注意不同版本可能使用 .agents/skills 或 .codex/skills，不要混淆。命令要说明应该输入到“Codex 对话框”还是“PowerShell 终端”。

内容用中文解释，关键术语附英文；采用适合个人操作手册的 Markdown 格式，包含必要的步骤、对比表和可复制示例，方便以后排成 US Letter 大小打印。

先在回复中给出完整介绍。这次只编写教程，不实际安装 skill、不修改环境或项目文件。

把提示词也放入文档中，这个文档按目前顺序给个序号

The subsequent instruction was to create 09-main-...md directly. This chapter, its bilingual companion, and their print editions implement that later instruction.

用户随后要求直接创建 09-main-...md。本章、对应双语文件及其打印版本，执行的是这条后续指令。

参考资料 (References)

1. <https://github.com/K-Dense-AI/scientific-agent-skills/tree/main/skills/scientific-visualization>
2. <https://learn.chatgpt.com/docs/customization/overview>
3. <https://learn.chatgpt.com/docs/skills-and-plugins>
4. <https://learn.chatgpt.com/docs/extend/mcp>
5. <https://learn.chatgpt.com/docs/build-skills>
6. <https://github.com/openai/skills>
7. <https://github.com/K-Dense-AI/scientific-agent-skills/blob/main/LICENSE.md>
8. <https://github.com/K-Dense-AI/scientific-agent-skills/tree/main/skills/scientific-visualization/scripts>
9. <https://github.com/openai/skills/blob/main/skills/.system/skill-installer/scripts/install-skill-from-github.py>
10. <https://code.visualstudio.com/docs/datascience/jupyter-notebooks>
11. <https://learn.chatgpt.com/docs/config-file/config-reference>
12. https://matplotlib.org/stable/api/_as_gen/matplotlib.figure.Figure.savefig.html
13. <https://matplotlib.org/stable/users/explain/text/fonts.html>
14. https://matplotlib.org/stable/users/explain/axes/constrainedlayout_guide.html